第 2 章　2 体問題を計算する

第 1 章では、原点に固定した中心質量の周りを動く 1 個の粒子を計算しました。この章では、二つの天体が互いに重力を及ぼし、両方とも動く問題へ進みます。時間積分には、前章で学んだ leapfrog 法の kick–drift–kick（KDK）形式を使います。新しく作る中心的な部分は、二つの位置から両方の加速度を求める処理と、重心・相対運動を用いた初期条件です。

ゼミで学んだ 2 体問題を、長い解析解の導出からやり直すことはしません。計算に必要な式を短く整理し、円・楕円・放物線の軌道を作ります。その後、全エネルギー、全角運動量、全運動量、重心運動を確かめます。最後に、指定した近点距離へ向かう接近軌道を作り、第 5 章の銀河相互作用へ向けた準備をします。

この章で学ぶこと

- 2 天体の重力加速度を、作用・反作用を保つ形で計算できます。
- 重心運動と相対運動を区別し、質量比に応じた初期位置・速度を設定できます。
- 第 1 章の KDK を使って、両天体を同じ時刻へ更新できます。
- 初期速度を変えて円・楕円・放物線を作り、解析的な軌道の性質と比較できます。
- 保存量と位置の誤差を使い、時間刻みの影響を確認できます。

2.1 一方を固定する近似を外す

2.1.1 本章で計算する対象を整理する

質量 M_1, M_2 の 2 天体を考えます。両方に有限の質量があり、互いに重力を及ぼします。外部からの力はなく、質量は時間によらず一定です。慣性系の x–y 平面内で運動を追跡します。

本章では、各天体を大きさのない**点質量**として扱います。これは、2 体問題の軌道と保存量を確かめるための練習モデルです。後の章で使う銀河の有限サイズ、円盤のテスト粒子、動力学的摩擦は、まだ入れません。とくに、2 天体が同じ位置へ衝突する初期条件は扱いません。

第 1 章では、中心天体が動かないと近似しました。今回はその近似を外すので、重い天体も動きます。質量が同じなら二つは対称に動き、質量が異なれば、重い天体ほど重心の近くを小さく動きます。まずは質量を異ならせ、この違いを見える形にします。

2.1.2 位置と相対位置を区別する

天体 $k = 1, 2$ の位置と速度を $\boldsymbol{R}_k, \boldsymbol{V}_k$ と書きます。大文字を使うのは、後の章の銀河中心と記法をそろえるためです。天体 1 から天体 2 へ向かう相対位置と、その変化率を

$$\boldsymbol{D} = \boldsymbol{R}_2 - \boldsymbol{R}_1, \qquad D = |\boldsymbol{D}|, \qquad \boldsymbol{U} = \boldsymbol{V}_2 - \boldsymbol{V}_1 = \frac{d\boldsymbol{D}}{dt} \tag{2.1}$$

と定義します。D は **2 天体の間の距離**です。それぞれの天体から原点までの距離とは、一般に一致しません。

位置の原点は、どちらかの天体へ毎回移動させるのではなく、まず一つの慣性系に固定します。天体 1 を原点に置いたような相対軌道の図は、計算後に式(2.1)で座標を引き算して作ります。座標を変換して表示することと、一方の天体の運動を無視することは別です。

2.2 二つの加速度を同じ位置から求める

2.2.1 力と加速度の違いを確認する

天体 1 は天体 2 へ、天体 2 は天体 1 へ引かれます。式(2.1)の向きを使うと、Newton の運動方程式は

$$M_1 \frac{d\boldsymbol{V}_1}{dt} = +GM_1M_2 \frac{\boldsymbol{D}}{D^3}, \tag{2.2}$$

$$M_2 \frac{d\boldsymbol{V}_2}{dt} = -GM_1M_2 \frac{\boldsymbol{D}}{D^3}. \tag{2.3}$$

両辺をそれぞれの質量で割り、加速度 $\boldsymbol{A}_k = d\boldsymbol{V}_k/dt$ を求めると、

$$\boldsymbol{A}_1 = +GM_2 \frac{\boldsymbol{D}}{D^3}, \qquad \boldsymbol{A}_2 = -GM_1 \frac{\boldsymbol{D}}{D^3} \tag{2.4}$$

となります。自分自身の質量は消えますが、重力を作る相手の質量は残ります。両方とも同じ時刻の $\boldsymbol{R}_1, \boldsymbol{R}_2$ から計算します。これらは Ryden の 2 体問題で用いる運動方程式に対応します。[1]

作用・反作用の関係は

$$M_1\boldsymbol{A}_1 + M_2\boldsymbol{A}_2 = \boldsymbol{0} \tag{2.5}$$

です。$M_1 \neq M_2$ なら、$\boldsymbol{A}_1 = -\boldsymbol{A}_2$ ではありません。例えば $M_1 = 2M_2$ なら、天体 2 の加速度の大きさは天体 1 の 2 倍です。

2.2.2 成分で確かめてから関数にする

$\boldsymbol{D} = (d_x, d_y)$、$D^2 = d_x^2 + d_y^2$ と書けば、例えば x 成分は

$$A_{1x} = \frac{GM_2 d_x}{(d_x^2 + d_y^2)^{3/2}}, \qquad A_{2x} = -\frac{GM_1 d_x}{(d_x^2 + d_y^2)^{3/2}} \tag{2.6}$$

です。y 成分も分子の d_x を d_y へ替えるだけです。天体 2 が天体 1 の右側にあれば、$A_{1x} > 0$、$A_{2x} < 0$ となります。この単純な配置で符号を確認してから、軌道を計算します。

実装では $G\boldsymbol{D}/D^3$ を一度だけ求め、それに相手の質量と符号を掛けます。2 体問題なので、全粒子対を探索する二重ループは必要ありません。この段階では、一般的な N 体重力計算へ広げません。

よくある失敗：同じ大きさになるのは力です

二つの加速度を単に符号反転して作る実装は、等質量の場合にしか正しくありません。等質量のテストだけでは、この誤りを見落とします。本章では $M_1 : M_2 = 2 : 1$ を標準にして、$M_1\boldsymbol{A}_1 + M_2\boldsymbol{A}_2$ を確認します。

2.3 重心運動と相対運動を分けて考える

2.3.1 重心が等速直線運動することを確かめる

全質量を $M_{\mathrm{tot}} = M_1 + M_2$ とし、質量で重み付けした平均位置を重心とします。

$$\boldsymbol{R}_{\mathrm{cm}} = \frac{M_1\boldsymbol{R}_1 + M_2\boldsymbol{R}_2}{M_{\mathrm{tot}}}, \qquad \boldsymbol{V}_{\mathrm{cm}} = \frac{M_1\boldsymbol{V}_1 + M_2\boldsymbol{V}_2}{M_{\mathrm{tot}}}. \tag{2.7}$$

全運動量は $\boldsymbol{P} = M_{\mathrm{tot}}\boldsymbol{V}_{\mathrm{cm}}$ です。式(2.2)と式(2.3)を足すと、内力が打ち消し合います。

$$\frac{d\boldsymbol{P}}{dt} = \boldsymbol{0}, \qquad \frac{d\boldsymbol{V}_{\mathrm{cm}}}{dt} = \boldsymbol{0}. \tag{2.8}$$

したがって、$t = 0$ からの重心の位置は

$$\boldsymbol{R}_{\mathrm{cm}}(t) = \boldsymbol{R}_{\mathrm{cm}}(0) + t\boldsymbol{V}_{\mathrm{cm}}(0) \tag{2.9}$$

です。重心を静止させた座標系を重心系と呼びます。本章の標準例では、初期重心を原点、重心速度を 0 にします。ただし、実装の検証には、重心に速度を与えた場合も使います。

2.3.2 相対運動を第 1 章と同じ形に書く

今度は、天体 2 の加速度から天体 1 の加速度を引きます。式(2.4)から

$$\frac{d\boldsymbol{D}}{dt} = \boldsymbol{U}, \qquad \frac{d\boldsymbol{U}}{dt} = -\mu\frac{\boldsymbol{D}}{D^3}, \qquad \mu \equiv GM_{\mathrm{tot}} \tag{2.10}$$

を得ます。右辺の形は、第 1 章の固定中心質量による運動と同じです。ただし、そこでは $\mu = GM$ だったのに対し、ここでは**二つの質量の和**が入ります。[1]

相対位置だけをこの式で計算する方法もあります。本章の主計算では両天体を直接動かし、相対運動はその結果から求めます。このようにすると、作用・反作用、重心、各天体の運動を同時に確認できます。相対運動の式は、初期条件や解析的な比較値を作るために使います。

2.3.3 相対位置から各天体の位置へ戻す

重心と相対位置が分かれば、式(2.7)と式(2.1)を連立して、二つの位置を求められます。速度にも同じ関係が成り立ちます。

$$\boldsymbol{R}_1 = \boldsymbol{R}_{\rm cm} - \frac{M_2}{M_{\rm tot}}\boldsymbol{D}, \qquad \boldsymbol{R}_2 = \boldsymbol{R}_{\rm cm} + \frac{M_1}{M_{\rm tot}}\boldsymbol{D},$$
$$\boldsymbol{V}_1 = \boldsymbol{V}_{\rm cm} - \frac{M_2}{M_{\rm tot}}\boldsymbol{U}, \qquad \boldsymbol{V}_2 = \boldsymbol{V}_{\rm cm} + \frac{M_1}{M_{\rm tot}}\boldsymbol{U}. \tag{2.11}$$

重心系では $\boldsymbol{R}_1, \boldsymbol{R}_2$ は逆方向です。それぞれの重心からの距離は

$$|\boldsymbol{R}_1| = \frac{M_2}{M_{\rm tot}}D, \qquad |\boldsymbol{R}_2| = \frac{M_1}{M_{\rm tot}}D \tag{2.12}$$

です。二つの軌道の大きさの比は、質量比の逆数になります。どちらも同じ相対運動から作られるので、束縛された 2 体軌道では周期は共通です。[1]

図2.1では、原点を重心とは別の場所に置きました。$\boldsymbol{D}$ は二つの位置ベクトルの差であり、重心からの距離でも原点からの距離でもありません。重い天体ほど重心に近いことも確認してください。

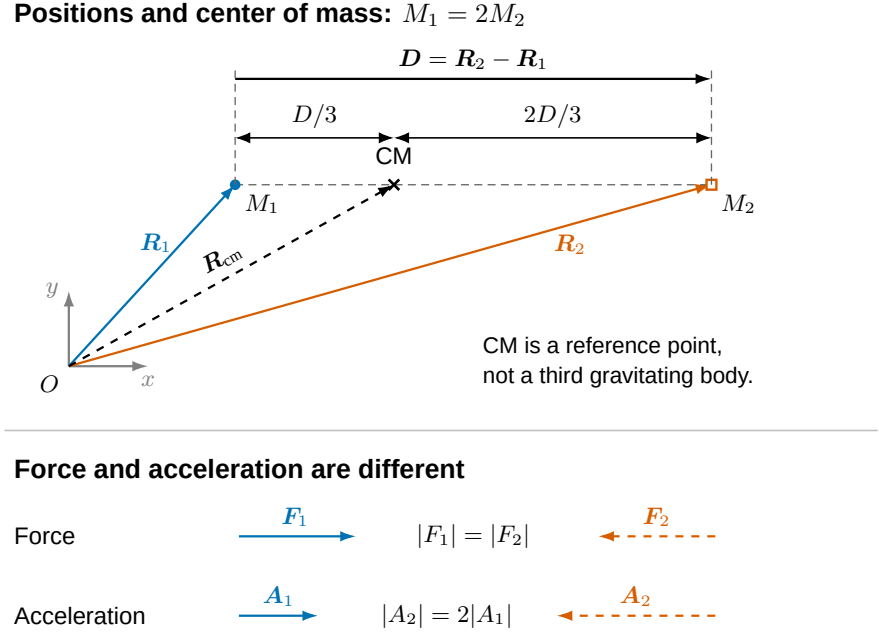


図 2.1 質量比 $M_1 : M_2 = 2 : 1$ の位置関係。任意の原点 O と重心 CM を区別しています。重心からの距離の比は 1 : 2 です。下段は力と加速度の違いで、力の大きさは等しくても加速度の大きさは異なります。段をまたいで矢印の長さを比較しません。

よくある失敗：単純な平均を重心にしない

$(\boldsymbol{R}_1+\boldsymbol{R}_2)/2$ が重心になるのは等質量のときだけです。同じく、$\boldsymbol{V}_1=-\boldsymbol{V}_2$ と置けば重心が静止するとは限りません。位置にも速度にも、式(2.11)の質量比を使います。

2.4 初期速度から軌道を選ぶ

2.4.1 円軌道を基準にする

相対距離が D_0 の円軌道を考えます。相対運動の向心加速度と重力加速度を等しくすると、相対円運動速度と周期は

$$v_{\rm c}=\sqrt{\frac{\mu}{D_0}},\qquad T_{\rm c}=2\pi\sqrt{\frac{D_0^3}{\mu}} \tag{2.13}$$

です。ここでの $v_{\rm c}$ は、一方の天体の速度ではなく、二つの**相対速度**の大きさです。重心系での各速度の大きさは、それぞれ $(M_2/M_{\rm tot})v_{\rm c}$、$(M_1/M_{\rm tot})v_{\rm c}$ です。

初期位置と速度を

$$\boldsymbol{D}(0)=(D_0,0),\qquad \boldsymbol{U}(0)=(0,\eta v_{\rm c}) \tag{2.14}$$

と置きます。η は接線方向の初期速度を調整する無次元の数です。ここでは動径方向の初期速度は 0 で、$\eta>0$ なら相対運動は反時計回りに始まります。

2.4.2 比エネルギーで軌道を分類する

相対運動の比エネルギーと比角運動量を

$$\varepsilon=\frac{U^2}{2}-\frac{\mu}{D},\qquad \ell_z=D_xU_y-D_yU_x \tag{2.15}$$

とします。どちらも式(2.10)で保存されます。式(2.14)の初期値を代入すると、

$$\varepsilon_0=\frac{\mu}{D_0}\left(\frac{\eta^2}{2}-1\right),\qquad \ell_{z,0}=\eta\sqrt{\mu D_0} \tag{2.16}$$

です。初期速度を変えることは、同じ距離から始める軌道のエネルギーと角運動量を変えることに対応します。

点質量の Kepler 問題では、$\varepsilon<0$ の非衝突軌道は楕円、$\varepsilon=0$ は放物線、$\varepsilon>0$ は双曲線です。円は楕円の特別な場合です。[1] 本章の 3 例を表2.1にまとめます。

$0<\eta<1$ では初期速度が円運動速度より小さいため、距離は初期位置からいったん減少します。動径速度が 0 だった初期位置は遠点です。$\eta=\sqrt{2}$ では脱出速度に相当し、初期位置から遠ざかって戻らない放物線になります。この例の放物線は、離れた場所から近づいてくる途中ではなく、**近点から出発する軌道**です。内向きに始める接近軌道は後で別に作ります。

表 2.1 式(2.14)を使う 3 例。D_0 と質量は共通で、変えるのは η だけです。

η	軌道	ε_0	初期位置の意味
1	円	$-\mu/(2D_0)$	半径一定
0.8	楕円	$-0.68\mu/D_0$	遠点
$\sqrt{2}$	放物線	0	近点

2.4.3 楕円の比較値を初期条件から求める

数値結果を見る前に、楕円の大きさと周期を求めておきます。Kepler 軌道の軌道長半径を a_{orb}、離心率を e とすると、

$$a_{\mathrm{orb}} = -\frac{\mu}{2\varepsilon}, \qquad e^2 = 1 + \frac{2\varepsilon \ell_z^2}{\mu^2} \tag{2.17}$$

です。[1] 2 次元なので、角運動量の大きさの 2 乗は ℓ_z^2 です。$0 < \eta < 1$ の初期条件を代入すると、

$$a_{\mathrm{orb}} = \frac{D_0}{2 - \eta^2}, \qquad e = 1 - \eta^2, \tag{2.18}$$

$$D_{\mathrm{a}} = D_0, \qquad D_{\mathrm{p}} = \frac{\eta^2 D_0}{2 - \eta^2}, \qquad T_{\mathrm{ell}} = 2\pi \sqrt{\frac{a_{\mathrm{orb}}^3}{\mu}}. \tag{2.19}$$

添字 a は遠点、p は近点です。a_{orb} は相対軌道の長半径であり、後の Plummer モデルのスケール長ではありません。

もう少し詳しく：軌道の形と時間の関係

楕円の形が分かっても、任意の時刻の位置を直ちに計算できるとは限りません。Kepler 方程式を解いて時刻と軌道上の位置を対応させる方法もありますが、本章ではその求根算法を追加しません。円の同時刻の解析解と、楕円の解析周期後の戻り位置を、独立な位置の確認に使います。詳しい軌道の記述は文献 [1] に譲ります。

2.5 Python で初期条件と重力を実装する

2.5.1 単位・配列・標準条件をそろえる

距離は kpc、時間は Gyr、質量は $M_\odot$ とし、内部速度を kpc Gyr^{-1} にそろえます。重力定数と速度の換算係数は第 1 章と同じです。以下では `import numpy as np` と `import matplotlib.pyplot as plt` を実行済みとします。

標準条件は本教材のために設定したもので、特定の実在天体の再現を目的としません。とくに、本章の天体は $10^{11} M_\odot$ 程度の質量を持っていても、銀河の内部構造を持たない点質量です。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
masses = np.array([1.0e11, 5.0e10])  # M1, M2 [Msun]
d0 = 20.0                            # 2 天体の初期距離 [kpc]
mtot = np.sum(masses)
mu = G * mtot
m_red = masses[0] * masses[1] / mtot
v_c = np.sqrt(mu / d0)               # 相対運動の円運動速度 [kpc/Gyr]
t_c = 2.0 * np.pi * np.sqrt(d0**3 / mu)
n_steps = 1000
t_end = t_c
dt = t_end / n_steps
eta_values = {"Circular": 1.0, "Elliptic": 0.8, "Parabolic": np.sqrt(2.0)}
```

表 2.2　本章の標準条件。3 ケースとも同じ T_{c} まで 1000 ステップで計算します。楕円の周期 T_{ell} とは区別します。数値は実行結果から生成しています。

量	値
M_1, M_2	$10^{11} M_\odot,\ 5\times10^{10} M_\odot$
D_0	$20\,\mathrm{kpc}$
v_{c}	$179.601851\,\mathrm{km\,s^{-1}}$
T_{c}	$0.684141\,\mathrm{Gyr}$
$\Delta t = T_{\mathrm{c}}/1000$	$0.684141\,\mathrm{Myr}$
η	$1,\ 0.8,\ \sqrt{2}$
$\boldsymbol{R}_{\mathrm{cm}}(0), \boldsymbol{V}_{\mathrm{cm}}(0)$	いずれも $\mathbf{0}$

`masses[0]` が M_1、`masses[1]` が M_2 です。位置配列 `pos` と速度配列 `vel` は形 `(2, 2)` で、行が天体、列が x, y 成分です。両者を横につないだ `state` は形 `(2, 4)` で、各行に `[x, y, vx, vy]` を入れます。第 1 章の約束を変えず、天体の行を 1 行から 2 行へ増やしています。

2.5.2 初期条件を関数にまとめる

式(2.11)を関数にします。入力の `separation` は相対位置ベクトル、`relative_velocity` は相対速度ベクトルです。どちらも 2 成分を渡します。重心位置・速度を省略したときは 0 とします。

```
def make_initial_state(masses, separation, relative_velocity, r_cm=None, v_cm=None):
    """ 相対位置・速度から 2 天体を配置します。本文 eq:ch2-inverse-transform。"""
    masses = check_masses(masses)
    separation = np.asarray(separation, dtype=float)
    relative_velocity = np.asarray(relative_velocity, dtype=float)
    r_cm = np.zeros(2) if r_cm is None else np.asarray(r_cm, dtype=float)
    v_cm = np.zeros(2) if v_cm is None else np.asarray(v_cm, dtype=float)
    for vector in (separation, relative_velocity, r_cm, v_cm):
        if vector.shape != (2,) or not np.all(np.isfinite(vector)):
            raise ValueError("Positions and velocities must be finite 2D vectors.")
    if np.linalg.norm(separation) == 0.0:
        raise ValueError("The two point masses must not coincide.")
    mtot = np.sum(masses)
    pos1 = r_cm - masses[1] / mtot * separation
    pos2 = r_cm + masses[0] / mtot * separation
    vel1 = v_cm - masses[1] / mtot * relative_velocity
    vel2 = v_cm + masses[0] / mtot * relative_velocity
    return np.column_stack((np.array([pos1, pos2]), np.array([vel1, vel2])))
```

`check_masses` は、二つの質量が有限かつ正であることを確かめる短い補助関数です。実装は Notebook に含めています。検証のための分岐と、初期値を作る物理の式を区別して読んでください。

$D_0 = 20\,\mathrm{kpc}$、質量比 $2:1$ では、重心系の初期位置は $(-20/3, 0)\,\mathrm{kpc}$ と $(40/3, 0)\,\mathrm{kpc}$ です。二つの距離を足すと $20\,\mathrm{kpc}$ になり、質量で重み付けした平均は 0 になります。

2.5.3 同じ粒子対を使って加速度を返す

最初に成分のまま計算し、向きと大きさを確認します。ここでは `state0` に上で作った初期値を入れたとします。

```
x1, y1 = state0[0, :2]
x2, y2 = state0[1, :2]
dx, dy = x2 - x1, y2 - y1
sep2 = dx**2 + dy**2
ax1, ay1 = G * masses[1] * dx / sep2**1.5, G * masses[1] * dy / sep2**1.5
ax2, ay2 = -G * masses[0] * dx / sep2**1.5, -G * masses[0] * dy / sep2**1.5
```

この初期位置で、天体 1 の x 加速度は約 $+562.31\,\mathrm{kpc\,Gyr^{-2}}$、天体 2 は約 $-1124.62\,\mathrm{kpc\,Gyr^{-2}}$ です。y 成分はいずれも 0 です。重い天体 1 の加速度が小さくなっています。

次は、同じ計算を 2 成分のベクトルでまとめます。

```
def acceleration_two_body(pos, masses):
    """2 天体の同じ時刻の位置から両方の加速度を返します。"""
    dr = pos[1] - pos[0]
    sep2 = np.dot(dr, dr)
    if not np.isfinite(sep2) or sep2 <= 0.0:
        raise ValueError("Point-mass separation must be finite and nonzero.")
    common = G * dr / sep2**1.5
    acc1 = masses[1] * common
    acc2 = -masses[0] * common
    return np.array([acc1, acc2])
```

`dr` が $\boldsymbol{D}$、`np.dot(dr, dr)` が D^2 です。返り値も形 `(2, 2)` にそろえます。分母へ小さな数を足して特異点を隠しません。点質量モデルで距離が 0 なら、その条件はこのコードの対象外として停止します。

確認：`masses[0] * acc[0] + masses[1] * acc[1]` が、丸め誤差の範囲で 0 になっているか確かめてください。これは軌道を積分する前にできる確認です。

2.6 KDK で両天体を同時に進める

2.6.1 積分器ではなく加速度関数を替える

時刻 t_n における 2 天体の状態が分かっているとします。KDK を、それぞれの天体 $k = 1, 2$ に対して

$$\begin{aligned} \boldsymbol{V}_k^{n+1/2} &= \boldsymbol{V}_k^n + \frac{\Delta t}{2}\boldsymbol{A}_k^n, \\ \boldsymbol{R}_k^{n+1} &= \boldsymbol{R}_k^n + \Delta t \boldsymbol{V}_k^{n+1/2}, \\ \boldsymbol{A}_k^{n+1} &= \boldsymbol{A}_k(\boldsymbol{R}_1^{n+1}, \boldsymbol{R}_2^{n+1}), \\ \boldsymbol{V}_k^{n+1} &= \boldsymbol{V}_k^{n+1/2} + \frac{\Delta t}{2}\boldsymbol{A}_k^{n+1} \end{aligned} \tag{2.20}$$

の順に行います。半整数ステップの速度は途中で使う値です。履歴には最後の kick を終えた、整数時刻の位置と速度を保存します。KDK の性質と導出は第 1 章で学んだ内容を使います。[5]

両天体の最初の kick を終えてから両天体を drift し、その**新しい 2 位置をそろえて**次の加速度を計算します。配列全体を更新することで、この順序を簡単に保てます。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, masses):
    """KDK。acc は現在の pos に対応する加速度です。"""
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, masses)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

第 1 章から変えたのは、最後の引数名を `mu` から `masses` へ替えた点です。今回は加速度関数に二つの質量を渡す必要があるためです。KDK の 4 行の計算と、加速度を返して再利用する構造は変わりません。

よくある失敗：片方だけを先に次の時刻へ進めない

天体 1 の位置を更新してから、その新しい位置と天体 2 の古い位置を使って天体 2 を更新すると、式(2.20)とは別の計算になります。同じ時刻に対する作用・反作用もそろいません。全天体の drift を終えてから、次の加速度を一度に求めます。

2.6.2 更新を繰り返して履歴を作る

最初の加速度だけをループの前で求めます。その後は、前ステップで返された加速度を次の最初の kick に使います。

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, masses):
    masses = check_masses(masses)
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, masses)  # 初回だけ計算します。
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, masses)
        history[n + 1] = np.column_stack((pos, vel))
        if not np.all(np.isfinite(history[n + 1])):
            raise FloatingPointError("Non-finite state; check the time step.")
    return times, history
```

`prepare_history` は第 1 章と同じ役割の補助関数ですが、今回は初期状態の形 `(2, 4)` を確認します。正のステップ数と最終時刻、有限の初期値を確認し、コピーと保存領域を用意します。履歴の形は `(n_steps + 1, 2, 4)` です。

`history[n, 1, :2]` は時刻 t_n における天体 2 の位置です。最初の軸が時間、次が天体、最後が位置・速度の成分であることを、確認してください。初期値も保存するので、出力時刻の数はステップ数より 1 つ多くなります。

本章でも、N_{step} ステップのための加速度関数の呼び出しは $N_{\mathrm{step}}+1$ 回です。1 回の呼び出しで**両方の加速度**を求めます。1000 ステップなら 1001 回です。粒子対の計算が 1 組なので、高速化手法や並列化は必要ありません。

2.6.3 3 ケースを同じ条件で計算する

各ケースの初期状態を作り直し、独立に積分します。一つの計算の最終状態を、次のケースの初期状態へ流用しないようにします。

```
cases = {}
start = perf_counter()
for name, eta in eta_values.items():
    initial = make_initial_state(masses, [d0, 0.0], [0.0, eta * v_c])
    times, history = integrate_leapfrog(acceleration_two_body, initial, t_end, n_steps,
    ↪  masses)
    cases[name] = {"eta": eta, "initial": initial, "times": times, "history": history}
```

本章の時間積分はすべて KDK です。RK4 や Euler 法との再比較は行いません。今回は積分法ではなく、初期条件、相対運動、保存量に注目します。

2.7 重心系と相対座標で軌道を見る

2.7.1 2 天体の軌道中心を確認する

まず $\eta = 0.8$ の楕円の場合に、二つの軌道を別々に描きます。青実線と丸が天体 1、朱色破線と四角が天体 2、黒い星印が重心です。両天体の軌道を同じ軸に重ねると、重い天体が重心の近くを動いていることが分かります。

```
history = cases["Elliptic"]["history"]
fig, ax = plt.subplots(figsize=(5.8, 5.2), layout="constrained")
ax.plot(history[:, 0, 0], history[:, 0, 1], color=BLUE, label="Body 1")
ax.plot(history[:, 1, 0], history[:, 1, 1], color=VERMILLION, ls="--", label="Body 2")
ax.plot(history[0, 0, 0], history[0, 0, 1], "o", color=BLUE, ms=6)
ax.plot(history[0, 1, 0], history[0, 1, 1], "s", color=VERMILLION, ms=6)
ax.scatter([0.0], [0.0], marker="*", color="black", s=100, label="Center of mass")
ax.set(xlabel="x [kpc]", ylabel="y [kpc]", title="Elliptic orbit; mass ratio 2:1")
ax.set_aspect("equal")
ax.legend(fontsize=9)
save_figure(fig, "ch02_barycentric")
```

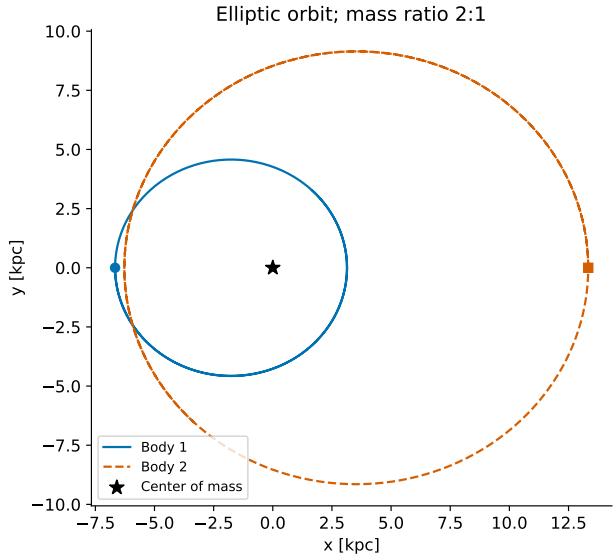


図 2.2　質量比 $M_1 : M_2 = 2 : 1$ の楕円軌道を重心系で表示した結果。丸・四角は初期位置です。二つの軌道は重心を共通の焦点とし、大きさの比は 1 : 2 です。計算時間は T_c で、楕円の約 1.586 周期に相当します。

原点に星印があるからといって、そこに第 3 の質量を置いたわけではありません。力を及ぼしているのは、動いている 2 天体だけです。重心は、二つの質量から定義する位置です。

Notebook ではこの履歴からアニメーションも作ります。画面の再生間隔を変えても、計算した軌道や物理的な時間は変わりません。本文の図も保存することで、動画なしでも軌道の違いを読めるようにします。

2.7.2 相対軌道の形を比較する

履歴の各時刻で天体 2 の位置から天体 1 の位置を引くと、相対軌道が得られます。式(2.10)を満たす軌道として、円・楕円・放物線を比べられます。

本章の接線速度から始める初期条件では、相対位置の極角を ϕ として、解析的な形を

$$D(\phi) = \frac{\eta^2 D_0}{1 + (\eta^2 - 1)\cos\phi} \tag{2.21}$$

と書けます。これは Kepler 軌道の式へ初期条件を代入し、$\phi = 0$ を初期位置に選んだものです。[1] $0 < \eta < 1$ ではその位置が遠点なので、通常の「近点を角度 0 にする」式と分母の符号が異なって見えます。

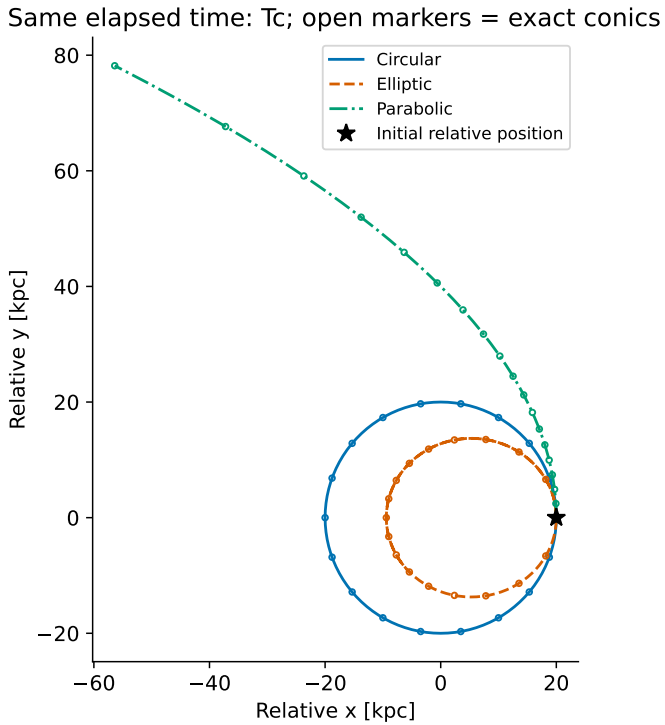


図 2.3 同じ D_0 から、初期接線速度だけを変えた相対軌道。3 ケースとも同じ T_c まで計算しています。白抜き丸は式(2.21)による解析的な軌道形状上の点で、等時間間隔の点ではありません。星印は共通の初期相対位置です。

図2.3の原点は相対座標での基準点です。この図だけでは、天体 1 が元の慣性系で静止しているようにも見えますが、実際の計算では図2.2のように両天体が動いています。

また、解析的な曲線に数値解が重なっても、時刻と位置の対応まで正しいとは限りません。軌道の形の確認と、同じ時刻における位置の確認を分けて行います。

2.8 2 体系の保存量を確かめる

2.8.1 全エネルギーと全角運動量を求める

第 1 章では 1 粒子の比エネルギーを計算しました。今回は二つの質量を掛けた**全エネルギー**を計算します。2 天体の運動エネルギーと、1 組の粒子対のポテンシャルエネルギーは

$$K = \frac{1}{2}M_1V_1^2 + \frac{1}{2}M_2V_2^2, \qquad W = -\frac{GM_1M_2}{D}, \qquad E = K + W \tag{2.22}$$

です。距離 D が変わると K と W はそれぞれ変わりますが、その和は一定です。例えば

$$\frac{dK}{dt} = -\frac{GM_1M_2}{D^3}\boldsymbol{D}\cdot\boldsymbol{U}, \qquad \frac{dW}{dt} = +\frac{GM_1M_2}{D^3}\boldsymbol{D}\cdot\boldsymbol{U} \tag{2.23}$$

となり、変化率が打ち消し合います。**片方の天体のエネルギーだけを保存量としない**ことが、第 1 章との違いです。

固定した原点の周りの全角運動量の z 成分は

$$L_z = \sum_{k=1}^{2} M_k(X_kV_{ky} - Y_kV_{kx}) \tag{2.24}$$

です。二つの内力は同じ直線上で逆向きに働くので、全トルクは 0 です。全角運動量も保存されます。全運動量 $\boldsymbol{P}$ と重心の等速直線運動は、式(2.8)、式(2.9)で確かめます。

> **よくある失敗：ポテンシャルエネルギーを 2 回足さない**
>
> 粒子対のエネルギーは $-GM_1M_2/D$ を 1 回だけ数えます。$M_1\Phi_2(\boldsymbol{R}_1) + M_2\Phi_1(\boldsymbol{R}_2)$ をそのまま足すと、同じ相互作用を 2 回数えてしまいます。各天体で計算してから足す書き方では、全体に $1/2$ が必要です。

2.8.2 全エネルギーと相対運動の比エネルギーを区別する

式(2.15)の ε と、式(2.22)の E は別の量です。この関係を整理するため、換算質量を

$$m_{\mathrm{red}} = \frac{M_1M_2}{M_{\mathrm{tot}}} \tag{2.25}$$

とします。式(2.11)を K へ代入すると、重心速度と相対速度の交差項が消え、

$$E = \frac{1}{2}M_{\mathrm{tot}}V_{\mathrm{cm}}^2 + m_{\mathrm{red}}\varepsilon \tag{2.26}$$

です。同じように、

$$L_z = M_{\mathrm{tot}}(\boldsymbol{R}_{\mathrm{cm}} \times \boldsymbol{V}_{\mathrm{cm}})_z + m_{\mathrm{red}}\ell_z \tag{2.27}$$

となります。重心系では、$E = m_{\mathrm{red}}\varepsilon$、$L_z = m_{\mathrm{red}}\ell_z$ です。[1]

$\mu = GM_{\mathrm{tot}}$ は重力の強さを表すパラメータ、m_{red} は質量です。同じ記号で呼ばないようにします。ε の単位は $\mathrm{kpc}^2\,\mathrm{Gyr}^{-2}$、$E$ はそれに $M_\odot$ を掛けた単位です。また、束縛・非束縛の分類に使うのは**相対運動の** ε です。系全体を速く動かしたときの、任意の慣性系での E の符号ではありません。

2.8.3 履歴から数値を求める

位置と速度は、KDK の最後の kick まで終えた同じ時刻の値を使います。以下では履歴から各天体の量を取り出し、全エネルギー、全角運動量、全運動量、重心をまとめて返します。

```
def diagnostics(history, masses):
    pos, vel = history[:, :, :2], history[:, :, 2:]
    dr, dv = pos[:, 1] - pos[:, 0], vel[:, 1] - vel[:, 0]
    separation = np.linalg.norm(dr, axis=1)
    kinetic = 0.5 * masses[0] * np.sum(vel[:, 0]**2, axis=1)
    kinetic += 0.5 * masses[1] * np.sum(vel[:, 1]**2, axis=1)
    potential = -G * masses[0] * masses[1] / separation
    energy = kinetic + potential
    momentum = masses[0] * vel[:, 0] + masses[1] * vel[:, 1]
    r_cm = (masses[0] * pos[:, 0] + masses[1] * pos[:, 1]) / np.sum(masses)
    angular = masses[0] * (pos[:, 0, 0] * vel[:, 0, 1] - pos[:, 0, 1] * vel[:, 0, 0])
    angular += masses[1] * (pos[:, 1, 0] * vel[:, 1, 1] - pos[:, 1, 1] * vel[:, 1, 0])
    return {"D": separation, "dr": dr, "dv": dv, "K": kinetic, "W": potential,
            "E": energy, "L": angular, "P": momentum, "Rcm": r_cm}
```

この関数は各保存量を保存するように補正するものではありません。得られた履歴を読み取り、値を計算するだけです。結果を見て毎ステップ重心や速度を修正すると、もとの積分器の誤差を評価できなくなります。

2.8.4 初期値が 0 の量は相対誤差にしない

重心系では $\boldsymbol{P}(0) = \boldsymbol{0}$ です。放物線では、理論的に $E(0) = 0$ でもあります。これらを分母にした「相対誤差」は定義できません。浮動小数点計算で初期エネルギーが小さな非零値になっていても、それで割るのは不適切です。

そこで、本章では初期距離と相対円運動速度から、共通の正の基準量を作ります。

$$E_{\mathrm{scale}} = \frac{GM_1M_2}{D_0}, \qquad P_{\mathrm{scale}} = M_{\mathrm{tot}}v_{\mathrm{c}}. \tag{2.28}$$

保存した全時刻にわたる最大値を使い、

$$e_E = \max_n \frac{|E^n - E^0|}{E_{\mathrm{scale}}}, \qquad e_L = \max_n \frac{|L_z^n - L_z^0|}{|L_z^0|},$$
$$e_P = \max_n \frac{|\boldsymbol{P}^n - \boldsymbol{P}^0|}{P_{\mathrm{scale}}}, \qquad e_{\mathrm{cm}} = \max_n \frac{|\boldsymbol{R}_{\mathrm{cm}}^n - \boldsymbol{R}_{\mathrm{cm}}^{\mathrm{exact}}(t_n)|}{D_0} \tag{2.29}$$

とします。$\boldsymbol{R}_{\mathrm{cm}}^{\mathrm{exact}}$ は式(2.9)です。本章の初期条件では $L_z^0 \neq 0$ なので、角運動量にだけその初期値を使えます。動径軌道など $L_z^0 = 0$ の問題では、角運動量にも別の基準量が必要です。

```
def conservation_errors(times, history, masses):
    values = diagnostics(history, masses)
    d_initial = values["D"][0]
    vc_initial = np.sqrt(G * np.sum(masses) / d_initial)
    e_scale = G * masses[0] * masses[1] / d_initial
    p_scale = np.sum(masses) * vc_initial
    r_cm_exact = values["Rcm"][0] + times[:, None] * values["P"][0] / np.sum(masses)
    err_e = np.max(np.abs(values["E"] - values["E"][0])) / e_scale
    l_scale = abs(values["L"][0])
    if l_scale == 0.0:
        raise ValueError("This angular-momentum normalization requires nonzero L(0).")
    err_l = np.max(np.abs(values["L"] - values["L"][0])) / l_scale
    err_p = np.max(np.linalg.norm(values["P"] - values["P"][0], axis=1)) / p_scale
    err_cm = np.max(np.linalg.norm(values["Rcm"] - r_cm_exact, axis=1)) / d_initial
    return np.array([err_e, err_l, err_p, err_cm])
```

表 2.3 3 ケースの保存量・重心の確認。いずれも T_c まで 1000 ステップの KDK です。エネルギーは $|E(0)|$ ではなく、式(2.28)で規格化しています。

軌道	e_E	e_L	e_P	e_{cm}
円	1.95×10^{-10}	2.55×10^{-15}	0	0
楕円	5.72×10^{-5}	5.42×10^{-15}	0	0
放物線	5.32×10^{-6}	4.87×10^{-15}	0	0

表2.3は、配布コードの実行結果です。表示された 0 は、この質量比と対称な初期配置をこの実装で計算したとき、対応する引き算が浮動小数点演算でも 0 になったことを意味します。任意の質量比・初期値で誤差が必ず 0 になる、という主張ではありません。後で重心を動かした場合も確認します。

2.8.5 保存量ごとに誤差が異なる理由を考える

KDK では、同じ位置から求めた二つの力が打ち消し合うため、kick は全運動量を保ちます。drift は速度を変えません。全角運動量についても、kick での全トルクが 0 で、drift の追加項は $\boldsymbol{V}_k \times \boldsymbol{V}_k = 0$ です。このため、本章の中心力の粒子対を正しく計算すれば、全運動量と全角運動量のずれは丸め誤差程度になります。

一方、KDK は全エネルギーを毎ステップ厳密に保存する方法ではありません。図2.4のように、有限の刻みによる差が現れます。全運動量と全角運動量がよく保存されていても、エネルギーや軌道の位相まで同じ精度で正しいとは限りません。

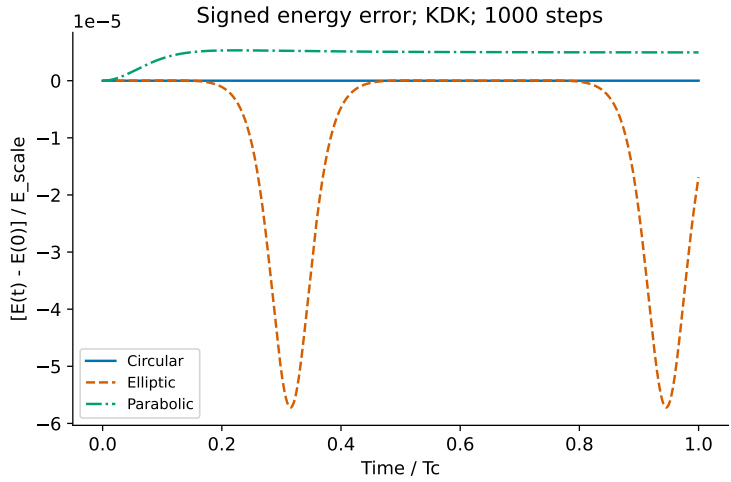


図 2.4 エネルギー差の時間変化。変化の向きも分かるよう、絶対値を取らない $[E(t) - E(0)]/E_{\rm scale}$ を描いています。楕円で差が大きくなる時期は、近点付近に対応します。

2.9 軌道の時間変化と刻みを確認する

2.9.1 近点と遠点を測る

式(2.18)と式(2.19)によれば、$\eta = 0.8$ の標準例は $e = 0.36$、$a_{\rm orb} = 14.705882\,\mathrm{kpc}$、$D_{\rm p} = 9.411765\,\mathrm{kpc}$ です。周期は $T_{\rm ell} = 0.431357\,\mathrm{Gyr}$ で、基準の円軌道周期 $T_{\rm c}$ より短くなります。

図2.5の円軌道では、距離はほぼ一定です。楕円では近点と遠点の間を往復し、放物線では近点から遠ざかります。楕円は $T_{\rm c}$ の間に 1 周以上するので、距離の谷が複数現れます。

標準計算で保存された楕円の最小距離は $9.412506\,\mathrm{kpc}$ でした。解析値との差には、時間積分の誤差に加えて、**真の近点通過時刻と保存時刻が一致しない**影響も含まれます。`np.argmin` で得るのは、保存された配列の最小要素です。連続時間の最小値そのものを解いているわけではありません。

2.9.2 保存量とは別に位置を比較する

円軌道には、第 1 章と同じ形の時刻に依存する解析解があります。

$$\boldsymbol{D}_{\rm circ}^{\rm exact}(t) = D_0\,(\cos\Omega t, \sin\Omega t)\,, \qquad \Omega = \sqrt{\frac{\mu}{D_0^3}}. \tag{2.30}$$

同じ時刻における数値相対位置との差を取り、

$$e_{\rm circ} = \max_n \frac{|\boldsymbol{D}^n - \boldsymbol{D}_{\rm circ}^{\rm exact}(t_n)|}{D_0} \tag{2.31}$$

を測ります。半径の差ではなく位置ベクトルの差なので、位相のずれも検出します。

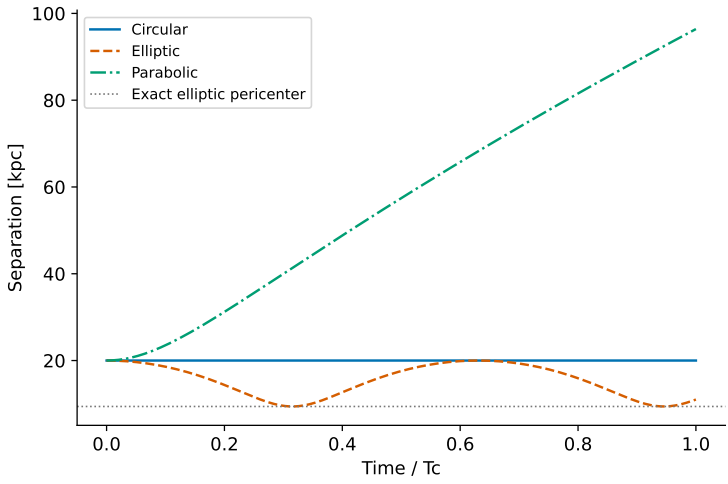


図 2.5 2 天体間距離の時間変化。横軸の T_c は 3 ケース共通の基準時間で、楕円の周期ではありません。点線は楕円の場合の解析的な近点距離です。

```
def exact_relative_circle(times, d0, mu):
    omega = np.sqrt(mu / d0**3)
    phase = omega * np.asarray(times)
    return np.column_stack((d0 * np.cos(phase), d0 * np.sin(phase)))
```

楕円では任意時刻の位置を解く代わりに、**解析的に求めた周期** T_{ell} を使います。ちょうど 1 周期後には初期位置へ戻るはずなので、

$$e_{\mathrm{ret}} = \frac{|\boldsymbol{D}^{N_{\mathrm{step}}} - \boldsymbol{D}(0)|}{D_0}, \qquad t_{N_{\mathrm{step}}} = T_{\mathrm{ell}} \tag{2.32}$$

を確認します。これは 1 周期後の戻り位置の誤差であり、途中も含めた最大誤差ではありません。数値解が戻った時刻を探してから、そこを周期と呼ぶ方法ではない点が重要です。

2.9.3 同じ最終時刻で刻みを細かくする

円軌道は T_c まで、楕円は T_{ell} まで計算し、それぞれの系列の中でステップ数だけを変えます。二つの系列の最終時刻は異なりますが、同じ系列では固定されています。二つの誤差の定義も異なるため、値の大小をそのまま「円と楕円の難しさ」の尺度にはしません。

```
step_counts = [100, 200, 400, 800]
convergence_rows = []
for count in step_counts:
    tc, hc = integrate_leapfrog(acceleration_two_body, cases["Circular"]["initial"], t_c, count, masses)
    dr_c = hc[:, 1, :2] - hc[:, 0, :2]
    exact_c = exact_relative_circle(tc, d0, mu)
    err_c = np.max(np.linalg.norm(dr_c - exact_c, axis=1)) / d0
    te, he = integrate_leapfrog(acceleration_two_body, cases["Elliptic"]["initial"], t_ell, count, masses)
    dr_end = he[-1, 1, :2] - he[-1, 0, :2]
    err_ell = np.linalg.norm(dr_end - np.array([d0, 0.0])) / d0
    err_energy_ell = conservation_errors(te, he, masses)[0]
    convergence_rows.append([count, t_c / count, t_ell / count, err_c, err_ell, err_energy_ell])
convergence = np.array(convergence_rows)
orders_c = np.log2(convergence[:-1, 3] / convergence[1:, 3])
orders_ell = np.log2(convergence[:-1, 4] / convergence[1:, 4])
```

表 2.4　KDK の時間刻み試験。円は解析的な 1 周期までの最大位置誤差、楕円は解析的な 1 周期後の戻り位置の誤差を示します。p は一つ前の行との誤差比から求めた推定次数です。

N_{step}	e_{circ}	p	e_{ret}	p
100	8.63×10^{-3}	—	9.13×10^{-3}	—
200	2.16×10^{-3}	1.9985	2.29×10^{-3}	1.9927
400	5.40×10^{-4}	1.9996	5.74×10^{-4}	1.9982
800	1.35×10^{-4}	1.9999	1.44×10^{-4}	1.9995

刻みを半分にしたときの誤差比から、

$$p = \frac{\log[e(\Delta t)/e(\Delta t/2)]}{\log 2} \tag{2.33}$$

を求めます。表2.4の最も細かい刻み側では、円が 1.9999、楕円が 1.9995 で、2 へ近づいています。第 1 章の 1 粒子の場合から 2 体へ変えても、KDK の 2 次精度を確認できます。

近点付近では距離が小さくなり、速度と加速度が大きく変わります。同じ刻みでも、円軌道より厳しい条件になることがあります。離心率をさらに大きくするときや、近点距離を小さくするときには、その新しい条件で刻み試験を行います。標準条件で得た誤差を、そのまま別の軌道の誤差とはみなしません。

もう少し詳しく：近点通過時刻の決め方

保存された距離の最小値を使う以外に、距離を時間の関数として補間したり、動径速度 $\boldsymbol{D}\cdot\boldsymbol{U}/D$ が 0 になる時刻を探したりする方法もあります。本章では新しい補間法・求根法を追加せず、最小値が保存時刻に依存することを明示します。補間と求根の基本は、第 3・4 章で別の具体例から学びます。

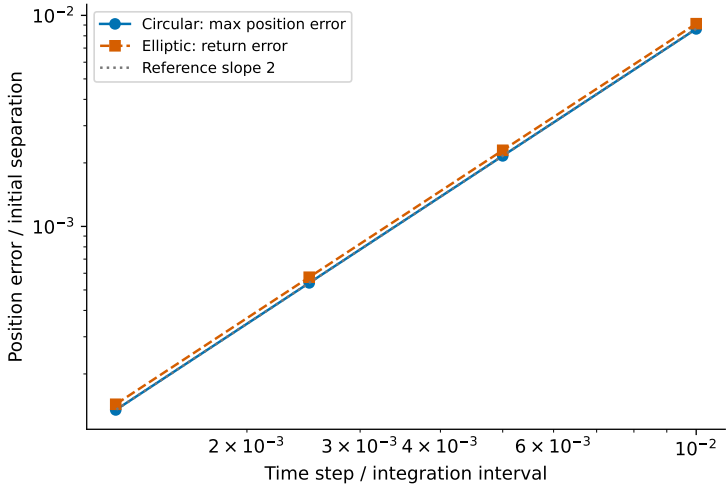


図 2.6　時間刻みと位置誤差の両対数表示。横軸は各系列の積分区間に対する刻みの比で、どちらも $1/N_{\mathrm{step}}$ です。点線は円の最も細かい刻みの点を通る傾き 2 の参照線です。数値データへの当てはめではありません。

2.10 近点距離を指定して接近させる

2.10.1 遠くから内向きに始める

これまでの放物線は、動径速度 0 の近点から始めました。銀河相互作用の準備では、離れた位置から互いに近づく初期条件が便利です。ここでは初期距離 D_0 と目標近点距離 D_p を指定し、$0 < D_\mathrm{p} < D_0$ とします。

放物線では $\varepsilon = 0$ です。近点では速度は接線方向だけなので、式(2.15)から

$$U_\mathrm{p}^2 = \frac{2\mu}{D_\mathrm{p}}, \qquad \ell_z = D_\mathrm{p} U_\mathrm{p} = \sqrt{2\mu D_\mathrm{p}} \tag{2.34}$$

です。角運動量保存より、初期距離での接線速度は $U_t = \ell_z / D_0$ です。初期位置でもエネルギー 0 とし、内向きの符号を選ぶと、

$$U_t = \frac{\sqrt{2\mu D_\mathrm{p}}}{D_0}, \qquad U_r = -\sqrt{\frac{2\mu}{D_0} - U_t^2} \tag{2.35}$$

になります。初期位置を $\boldsymbol{D} = (D_0, 0)$ とすれば、$\boldsymbol{U} = (U_r, U_t)$ をそのまま渡せます。ここでも原点を重心にした各天体の初期値へ、式(2.11)で変換します。

```
def make_parabolic_encounter(masses, d_start, d_peri):
    masses = check_masses(masses)
    if not (np.isfinite(d_start) and np.isfinite(d_peri) and 0.0 < d_peri < d_start):
        raise ValueError("Require finite distances with 0 < d_peri < d_start.")
    mu = G * np.sum(masses)
    ell = np.sqrt(2.0 * mu * d_peri)
    vt = ell / d_start
    vr = -np.sqrt(2.0 * mu / d_start - vt**2)
    return make_initial_state(masses, [d_start, 0.0], [vr, vt])

encounter_initial = make_parabolic_encounter(masses, 60.0, 8.0)
t_enc, h_enc = integrate_leapfrog(acceleration_two_body, encounter_initial, 0.60, 1200,
↪  masses)
enc_values = diagnostics(h_enc, masses)
i_peri = np.argmin(enc_values["D"])
```

2.10.2 指定した近点の再現性を確かめる

この追加実験では、質量は標準例のまま、$D_0 = 60\,\mathrm{kpc}$、$D_\mathrm{p} = 8\,\mathrm{kpc}$、最終時刻 $0.60\,\mathrm{Gyr}$、1200 ステップとします。刻みは $0.5\,\mathrm{Myr}$ です。3 ケース比較の D_0 や T_c と混同しないよう、条件を分けて記録します。

得られる初期相対速度は、$U_r = -136.518\,\mathrm{km\,s^{-1}}$、$U_t = 53.547\,\mathrm{km\,s^{-1}}$ です。内向きの動径速度が負であることを確認してください。計算後に保存された最小距離は $8.000330\,\mathrm{kpc}$、その保存時刻は $0.3145\,\mathrm{Gyr}$ でした。指定した $8\,\mathrm{kpc}$ へ近づき、その後は離れていきます。

近づいた後に離れていくことは、計算の失敗ではありません。摩擦のない点質量 2 体では相対エネルギーが保存されるため、放物線遭遇が自動的に束縛軌道へ変わることはありません。近接通過と合体は区別します。

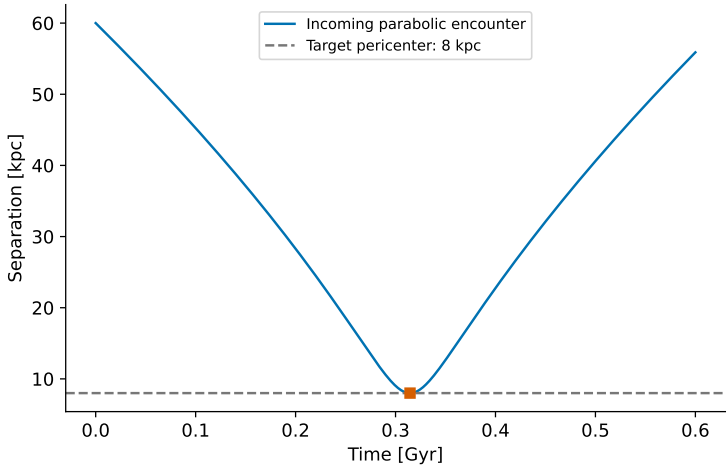


図 2.7　点質量 2 体の内向きに始まる放物線遭遇。破線は指定した近点距離、四角は保存された距離の最小値です。これは本章の質量比 2 : 1 の計算であり、後の等質量 Plummer 銀河の標準実験とは別の条件です。

本章の式(2.35)は、ポテンシャルが $-\mu/D$ の場合の式です。第 5 章で有限サイズを表す中心間重力へ替えたとき、μ だけを同じにしてこの速度を使えば、同じ近点距離が得られるとは限りません。**重力のモデルを変更したら、そのモデルに対応するエネルギーと初期条件を使う**という原則を引き継ぎます。

2.11 条件を一つずつ変えて確かめる

2.11.1 重心を等速で動かす

計算の原点や、系全体の等速運動を変えても、相対運動は変わらないはずです。式(2.11)の重心位置を $(5, -3)\,\mathrm{kpc}$、重心速度を $(30, -20)\,\mathrm{km\,s^{-1}}$ として、同じ楕円を計算します。

```
r_boost = np.array([5.0, -3.0])
v_boost = np.array([30.0, -20.0]) * KM_S_TO_KPC_GYR
boost_initial = make_initial_state(masses, [d0, 0.0], [0.0, 0.8 * v_c], r_boost, v_boost)
t_boost, h_boost = integrate_leapfrog(acceleration_two_body, boost_initial, t_c, n_steps,
    masses)
bv = diagnostics(h_boost, masses)
base = diagnostics(cases["Elliptic"]["history"], masses)
boost_relative_error = np.max(np.linalg.norm(bv["dr"] - base["dr"], axis=1)) / d0
boost_errors = conservation_errors(t_boost, h_boost, masses)
```

相対軌道の差を D_0 で割った最大値は 4.87×10^{-14}、重心が式(2.9)の直線から外れる量は D_0 単位で 4.06×10^{-15} でした。これらは本実装での丸め誤差程度の差です。重心系の計算でたまたま誤差が 0 になった場合だけでなく、移動する重心でも確認する意味があります。

このとき、任意の原点の周りの E や L_z の初期値は、重心系と同じとは限りません。比較すべきなのは、それぞれの系で保存する量と、式(2.26)・式(2.27)の関係です。

2.11.2 全質量を固定して質量比を変える

次は $M_{\rm tot}, D_0, \boldsymbol{U}(0)$ を固定し、$q = M_2/M_1$ だけを変えます。相対運動の式に現れるのは $\mu = GM_{\rm tot}$ なので、相対軌道は同じです。しかし、各天体が重心から離れる距離は変わります。

表 2.5 全質量を固定した質量比の比較。相対軌道の差は、標準の $q = 0.5$ との差の最大値です。初期相対速度は $0.8v_{\rm c}$ に固定しています。

$q = M_2/M_1$	$\lvert\boldsymbol{R}_1(0)\rvert$ [kpc]	$\lvert\boldsymbol{R}_2(0)\rvert$ [kpc]	相対軌道の差 $/D_0$
1.0	10.000000	10.000000	1.57×10^{-14}
0.5	6.666667	13.333333	0
0.1	1.818182	18.181818	4.04×10^{-14}

M_1 を固定したまま M_2 を変えると、$M_{\rm tot}$ も変わるので別の実験です。パラメータを変えたときには、何を固定し、何が連動して変わるかを記録します。これは銀河相互作用の比較実験にも必要な習慣です。

2.12 本章のまとめ

本章では、互いに重力を及ぼす 2 天体を KDK で動かしました。第 1 章から積分法を作り直すのではなく、2 天体の同じ時刻の位置から加速度を返す関数へ替えました。最後に評価した加速度を再利用し、両天体の位置と速度を整数時刻にそろえて保存しました。

重心は全質量による重み付き平均です。相対運動には $\mu = G(M_1 + M_2)$ が入り、各天体の軌道は重心運動と相対運動から作られます。初期接線速度を変えると、円・楕円・放物線を作れることも確認しました。

保存量は、今回は 2 体系の全エネルギー・全角運動量・全運動量です。放物線のように初期エネルギーが 0 の問題では、その値で誤差を割りません。保存量だけでなく、円の同時刻の位置、楕円の 1 周期後の戻り位置、刻みを変えた収束も確かめました。

最後に、初期距離と近点距離から内向きの接近軌道を作りました。ただし、本章の重力源はまだ点質量です。第 3 章では、質量に広がりをもつ球対称の Plummer モデルを導入し、その中の星の運動へ進みます。点質量の式と広がった銀河の式を、混同しないことが重要です。

演習

演習 2.1　確認：重心と作用・反作用　$M_1 = 10^{11} M_\odot$、$M_2 = 5 \times 10^{10} M_\odot$、$D_0 = 20\,\mathrm{kpc}$ の円軌道を重心系に置いてください。初期位置と各天体の速度を求め、全運動量が 0 になることを示してください。また、初期加速度の大きさの比と、力の大きさの比を求めてください。

演習 2.2　確認：楕円の形と周期　同じ D_0 で $\eta = 0.8$ としたとき、初期位置が遠点である理由を説明してください。$e, a_{\mathrm{orb}}, D_{\mathrm{p}}, T_{\mathrm{ell}}$ を初期条件から求め、数値計算の距離の時間変化と比較してください。保存された最小距離と解析的な近点距離が完全には一致しない理由も述べてください。

演習 2.3　確認：放物線のエネルギー誤差　$\eta = \sqrt{2}$ の計算で、$|E(t) - E(0)|/|E(0)|$ を誤差に使わない理由を説明してください。本章の E_{scale} で規格化すると、どの物理的な大きさに対するずれを測ることになるでしょうか。

演習 2.4　応用：全質量を固定する　表2.5と同じ条件で $q = 1, 0.5, 0.1$ を比較してください。各天体の重心からの距離が変わっても、相対軌道が同じになることを確認してください。M_1 だけを固定して q を変えた場合と、何が異なるか説明してください。

演習 2.5　応用：移動する重心　重心に位置 $(5, -3)\,\mathrm{kpc}$、速度 $(30, -20)\,\mathrm{km\,s^{-1}}$ を与えてください。重心が等速直線運動し、相対軌道がもとの楕円と同じになるか確かめてください。任意の慣性系での全エネルギー E の符号だけでは、束縛軌道かどうかを判断できない理由も説明してください。

演習 2.6　応用：KDK の時間反転　標準の楕円を T_{c} まで計算し、その最終位置を保ったまま、**両天体の速度**を符号反転してください。同じ正の時間刻みでさらに T_{c} だけ進めたとき、初期位置と初期速度の符号反転へ戻るか確認してください。時間反転の試験に通ることだけで、軌道の位相誤差が小さいと言えるでしょうか。

演習 2.7　発展：脱出速度を超える軌道　$\eta = 1.6$ として、ほかは 3 ケース比較と同じ条件で計算してください。相対エネルギーの符号と離心率を求め、軌道が双曲線になることを確認してください。T_{c} 後の距離を放物線の場合と比べ、全エネルギーの誤差も確認してください。積分器の変更は必要ありません。